

Review of Your August 2nd Plan

Verified against the repo at `(9767e6b)` and the paper. Verdict: **execute 8 of 9 items as written**. One item (slide 2) will zero out the attention branch — resolve that first.

THE ONE TRAP — Slide 2: `(gamma = 1.0)` for the original view

What your plan does

"since we are in GInfoMinMax main model so there is no augmentation here, resulting that gamma will be 1 we can explicitly give that."

What that produces

`TA_encoder.py:308` clamps to `[eps, 1-eps]` with `(eps = 1e-4)`, so `(gamma = 0.9999)`. Then:

```
python

if self.training:
    lambda_ = Beta(1 - gamma, gamma).sample() # Beta(1e-4, 0.9999) -> mass at
else:
    lambda_ = 1 - gamma                        # = 1e-4
```

```
python

x = x + lambda_ * x_norm * x_trans
    = x + 0.0001 * x_norm * x_trans
```

Attention contributes 0.01% of x's magnitude. That is zero.

And in training too — `(Beta(1e-4, 0.9999))` puts essentially all its mass at 0.

Why this matters more than it looks

`(get_embeddings())` encodes the **original** graph. So the vectors your SVM classifies come from `(x_topo)` alone — a plain GCN.

The TAE attention branch is the paper's headline contribution. Tables 7, 8 and 9 exist solely to show TAE improving AGCL, AD-GCL and LightGCL. Under your plan it never reaches the classifier in either training or inference. You would be reproducing "GraSTI-ACL minus the TAE" and reporting it as GraSTI-ACL.

Where the ambiguity actually is

Eq. 18 is $\lambda = B(\gamma, 1-\gamma)$. Your code uses `(Beta(1- $\gamma$ ,  $\gamma$ ))` — reversed — justified in the comment by §3.3's prose. **Both readings are defensible, but they give opposite answers for the original view:**

Ordering	$\gamma_{\text{orig}} = 1 \rightarrow \lambda_{\text{orig}}$	$\gamma_{\text{aug}} = 0.5 \rightarrow \lambda_{\text{aug}}$	Attention at inference
Reversed $\text{Beta}(1-\gamma, \gamma)$ (your code)	0.0	0.5	dead
Literal $\text{Beta}(\gamma, 1-\gamma)$ (Eq. 18)	1.0	0.5	full

One more fact worth knowing: $\text{Beta}(1-\gamma, \gamma)$ has **both** shape parameters below 1 for every $\gamma \in (0,1)$, so it is always U-shaped. The paper says it chose $(x=\gamma, y=1-\gamma)$ specifically to avoid Mixup's $(0.4, 0.4)$ which "bias the weight towards two extremes" — but $\text{Beta}(\gamma, 1-\gamma)$ is U-shaped too, and more extreme than $\text{Beta}(0.4, 0.4)$ near the ends. **The paper's stated motivation does not match its own equation.** You are not going to resolve this by re-reading it.

What to do

Do **not** silently ship $\gamma_{\text{orig}} = 1.0$ with the reversed ordering. Pick one:

Option A — literal Eq. 18. Switch to $\text{Beta}(\gamma, 1-\gamma)$. Then $\gamma_{\text{orig}} = 1 \rightarrow \lambda = 1$ (full attention on the original view), $\gamma_{\text{aug}} = \text{mean}(A') \rightarrow \lambda_{\text{aug}} = \text{mean}(A')$. Equation- faithful, attention alive on the classification path. Contradicts the §3.3 prose.

Option B — keep reversed, set $\gamma_{\text{orig}} = \text{mean}(|W|)$. This is what the current fallback already computes ($\approx 0.39 \rightarrow \lambda_{\text{orig}} \approx 0.61$). Defensible if you read "average of all elements in the adjacency matrix" as the *weighted* adjacency actually fed to the GCN, which after your Issue-4 fix is $|W|$. Prose-consistent, attention alive.

Recommendation: run both. It is one flag and one run each. Log λ_{orig} per epoch so you can see it is non-zero. Then report whichever you use as an explicit interpretation of an equation the paper states inconsistently — that is a legitimate methods sentence, and it is far better than discovering at write-up that your TAE never fired.

Whatever you choose, add this assertion so the failure can never be silent again:

```
python
assert lambda_ > 1e-3, f"attention branch is dead: lambda={lambda_}, gamma={gam
```

WHAT IS CORRECT — execute as written

Slide 5 (Issue 4, $|W| \cdot A'$). Your arithmetic is right: $0.39 \times 0.5 \approx 0.2$, weaker and flatter. A' already encodes $|W|$ through ToyNet, so multiplying again double-counts. Augmented view gets $\text{batch_aug_edge_weight}$ alone. ✓

Slide 6 (1 GCN layer). Reasoning is exactly right — on a complete graph one layer already reaches every node, so layer 2 buys nothing and costs oversmoothing. ✓

Slide 7 (emb_dim). Correct catch that with 1 layer, `(layerwise) == (standard)`, so 128 gives 128 not 256. Testing 128 and 256 is the right response. ✓

Slide 8 (normalization). Your scale-mismatch concern is real — **and the code already solves it.** Lines 293 and 337:

```
python

x_trans = F.normalize(x_trans, dim=1) # direction only
x_norm = torch.sqrt(torch.sum(x**2, dim=1, keepdim=True) + 1e-12)
x = x + lambda_ * x_norm * x_trans # scaled to x's own
```

⊗ keeps its ALFF-derived magnitude; attention is scaled *relative to it*, so λ controls the ratio regardless of the branches' absolute scales. **This item is already done — do not change it.** ✓

Slide 13 (unshare the backbone). Correct, and the decisive argument is the one in your deck: with shared tensors, `(opt_shared.step())` runs after `(-view_loss).backward()` at line 272 and after `(model_loss.backward())` at line 356 — ascent then descent on the same objective, over the same parameters. `(min_θ max_θ L(θ))` is degenerate. Two independent `(TAEncoder) + (ToyNet)` pairs, two optimizers. ✓

Slide 9 vs 13. You caught the conflict yourself — slide 13 supersedes slide 9. After unsharing there are no shared tensors, so the three-optimizer split is unnecessary. Delete `(opt_shared)` and the `(model.encoder.train()) / (model.net.train())` re-assertions after `(model.eval()) / (view_learner.eval())` — those only existed to undo the sharing side effect and will confuse you later. ✓

Slides 11/12 ((edge_logits) → (edge_logits_v1)). You wrote "both have the same recipe so it is not a big issue." **That was true while the backbone was shared. It stops being true the moment you do slide 13** — they become genuinely different networks, and Eq. 13's `(L_CE(A_ij, M_ij))` needs the adjacency that actually built the augmented graph. Slide 12 says it is already applied. Verify it is still there after the unshare. ✓

Slide 10 (dead (beta)). Correct. Repurpose the parameter slot rather than deleting five signatures. ✓

SLIDE 4 — passing `(gcn_w)` to the view learner: approve, with one note

Currently `(view_learner(..., gcn_edge_weight=None, ...))`, so its encoder sees an unweighted complete graph while the model's sees `([w])`. Making them consistent is right.

Note for your methods section, not a blocker: `(edge_prod)` (line 45) is `(sum(node_emb_src * node_emb_dst))` — the dot product of **encoder outputs** (32-dim). Eq. 4 defines `(M_ij = σ(v_i · v_j))` where `(v_i, v_j ∈ R^d)` are "nodal feature vectors, composed of ALFF features from

three frequency bands" — i.e. **d = 3, the raw ALFF**. Your version is a reasonable practical implementation, but it is a deviation from Eq. 4 and should be disclosed.

SLIDE 15 — your reg experiment, read carefully

You measured: no reg → drop ratio 0.9, `calc_loss` ≈ 3.6. With `reg_lambda = 1.0` → drop falls to 0.5, `calc_loss` barely moves.

Random baseline for batch 32 is 3.43. So at 3.6 the encoder is still *slightly worse than chance* at matching a subject to its own augmented view. reg fixed the graph destruction; it did not make the encoder learn.

That is the expected result, because you measured it **before** the unshare. While the backbone is shared, the encoder is fighting itself — no regularizer can fix that.

Re-measure `reg_lambda` after slide 13, not before. The arms race is a different game once Φ and Θ are separate parameter sets, and 1.0 may no longer be the right value.

SLIDE 14 — "Decision: ????"

Your framing: the view learner drops edges to make the graph harder, but in doing so destroys the information the encoder needs.

That tension is not a bug — it is the method. Eq. 15's `min_Φ max_Θ` is exactly this fight, and the thing that keeps it balanced is the regularizer from slide 15. So the answer to slide 14 is slide 15: `reg_lambda` is the knob. Tune it by watching `fin_reg`.

<code>fin_reg</code>	Meaning	Action
→ 0.9	augmenter destroying the graph	raise <code>reg_lambda</code>
0.2 - 0.5	healthy	leave it
→ 0.0	no augmentation happening	lower <code>reg_lambda</code>

EXECUTION ORDER

```
Run 0  Slide 13 (unshare) + slide 11/12 (edge_logits_v1) + slide 5 (A'
      alone)
      + slide 4 (gcn_w to view learner) + gamma decision from above
      -> the adversarial loop is either alive or it isn't

Run 1  Slide 6 (--num_gc_layers 1) + slide 7 (--emb_dim 128, then 256)
      + --batch_size 128

Run 2  Re-tune reg_lambda on the post-unshare dynamics
```

The three numbers to log every epoch

1. `calc_loss` separately from `model_loss`. Random = 3.43 at batch 32 ($= \log(\text{batch_size} - 1)$). If you move to batch 128 the baseline becomes 4.84 — recompute it, do not compare across batch sizes.
2. `fin_reg` — the drop ratio. Table above.
3. `lambda_` for the original view — must not be ~ 0 , or the TAE is dead.

`calc_loss` falling below its baseline is the pass/fail gate. Above it, the model is worse than chance at recognising its own augmented view, and no downstream classifier can recover from that. Do not proceed to Run 1 until Run 0 clears it.

ONE HONEST CAVEAT

I am confident about the mechanism in each item above — each is a specific line with a consequence I traced. I am not able to promise the accuracy number that results. The γ question in particular is a genuine fork the paper does not settle, which is why I am telling you to run both rather than picking for you.